

Orchestration Framework Recommendations for Four Scenarios				
Scenario	Need / Context	Recommended Orchestration / Framework	One-line Why	Decision Factors
Scenario 1 — The understaffed marketing team	Customer-support triage bot for refund, shipping, and product questions. No engineering capacity. Working version needed this week. Ops staff must tweak routing logic without code.	Flowise	Use Flowise because it is visual, drag-and-drop, low/no-code and is best for business users who need fast prototyping without engineering support.	Non-technical business users → no/low-code; prototype needed quickly; simple routing logic; maintainable by ops team.
Scenario 2 — The research-brief assembly line	Automate a repeatable market-research brief workflow: researcher gathers sources, analyst synthesizes, writer drafts, editor polishes. Python developers are available, but they do not want to hand-build orchestration plumbing.	CrewAI	Use CrewAI because it supports role-based multi-agent workflows where agents have clear roles, goals, and tasks with structured handoffs.	Multiple agents; predefined roles; role → task → handoff pattern; structured business-process automation; ship fast in code.
Scenario 3 — The self-debugging data analyst	Prototype an agent that writes Python data-analysis code, runs it, reads errors, and rewrites until it works. A coder agent and critic agent must converse for an unknown number of turns. Sandbox execution is needed.	AutoGen (Microsoft)	Use AutoGen because it is conversational, event-driven, multi-agent and strong for iterative code-execution loops with unknown back-and-forth turns.	Agents must talk to each other; open conversation; unknown number of turns; code execution loop; coder and critic collaboration.
Scenario 4 — The regulated enterprise platform	Production clinical-knowledge assistant for healthcare. Requirements include RAG over many internal data sources, custom branching, human-in-the-loop approvals, pause/resume, checkpointing, tracing, observability, and compliance audits. Dedicated engineering team owns the system.	LangChain + LangGraph	Use LangChain + LangGraph because it provides maximum control, integrations, stateful graphs, cycles, checkpoints, human-in-the-loop gates, and production tracing for complex regulated systems.	Engineering-owned production system; complex custom branching; RAG integrations; stateful workflows; checkpointing; observability and tracing; human-in-the-loop approval gates.